Lab 5: Global State Management

Part 1: Practical Tasks

In this lab, you will use **Zustand** to manage a simple global UI state (Theme Toggle) and **Redux Toolkit** to manage a complex data state (The Shopping Cart) and **ContextAPI** for Localization

Task 1: Environment Setup

Before beginning, open your terminal (e.g., Git Bash), navigate to your project directory, and run the following command to install the required dependencies:

```
npm install @reduxjs/toolkit react-redux zustand
```

Task 2: Implementing Zustand (Global UI Theme)

1. **Create the Store:** In your src folder, create a new folder named store and a file named useThemeStore.js.
2. **Define the Hook:** Create a Zustand store that contains a theme state (defaulting to 'light') and a toggleTheme function that switches the state between 'light' and 'dark'.
3. **Consume the Store:** Open your MainLayout or Navbar component. Import your useThemeStore. Add a button to the Navbar that, when clicked, triggers the toggleTheme function. Conditionally apply a CSS class or UI library property to change the background color of your app based on the active theme.

Task 3: Implementing Redux Toolkit (The Shopping Cart Slice)

1. **Create the Slice:** Inside your store folder, create a file named cartSlice.js.
2. **Configure Initial State:** Define an initial state with an empty array: initialState: { cartItems: [] }.
3. **Build Reducers:** Create two reducers inside the slice:
   - addToCart: Takes a product object payload and pushes it into the cartItems array.
   - removeFromCart: Takes a product ID payload and filters it out of the cartItems array.
4. **Export:** Export the actions (addToCart, removeFromCart) and the reducer.

Task 4: Configuring and Providing the Redux Store

1. **Create the Store:** In the store folder, create store.js. Use configureStore from RTK to combine your cartSlice reducer into the main store.
2. **The Provider:** Open your main.jsx (or index.js). Import Provider from react-redux and your store. Wrap your entire <App /> component inside the <Provider store={store}> so every component has access to the Redux architecture.

Task 5: Dispatching Actions (Adding Products)

1. Open your ProductCard component.
2. Import useDispatch from react-redux and the addToCart action from your slice.
3. Initialize the dispatch function: const dispatch = useDispatch();
4. Add an "Add to Cart" button to the card. When clicked, it should execute dispatch(addToCart(productData)), sending the current product's details to the global store.

Task 6: Selecting State (Reading the Cart)

1. **The Cart Badge:** Open your Navbar component. Import useSelector. Use it to read the length of the cartItems array from the store. Display this number next to your Cart link (e.g., "Cart (3)").
2. **The Cart Page:** Open your Cart component (from Lab 1). Use useSelector to retrieve the entire cartItems array. Use **List Rendering** (.map()) to display all the products the user has added. Add a "Remove" button to each item that dispatches the removeFromCart action.

Task 7: Implementing Localization (Context API)

3. **Create the Context:** Inside your `src` folder, create a `context` folder and a file named `LanguageContext.jsx`. Use `createContext()` to initialize it.
4. **Build the Provider:** Create a component named `LanguageProvider` that accepts `children` as a prop. Inside it, declare a local state for `language` (defaulting to `'en'`) and a function to toggle it to `'ar'`.
5. **Return the Wrapper:** Return your `<LanguageContext.Provider>` passing the state and toggle function in the `value` prop, rendering `{children}` inside it. Export a custom hook (`useLanguage`) that calls `useContext(LanguageContext)` for easy access.
6. **Wrap the App:** Open `main.jsx`. Import your `LanguageProvider` and wrap your entire `<App />` inside it.
7. **Consume the Context:** Open your `Navbar`. Import `useLanguage`. Add a button to toggle the language. Use conditional rendering to display "Welcome" if the state is 'en', and "مرحباً" if the state is 'ar'.

Part 2: Technical Terminology

Before writing any implementation code, it is mandatory to understand the exact architectural problem we are solving and the tools we are using.

1. The Core Problem: Prop Drilling

- **Definition:** The process of passing data (props) from a top-level parent component down through multiple layers of intermediate child components that do not actually need the data, just to reach a deeply nested component that does.
- **The Architecture Flaw:** As your E-commerce app grows, passing a `cart` state from `App` -> `MainLayout` -> `Navbar` -> `CartIcon` becomes fragile and difficult to maintain.

2. Global State Management

- **Definition:** A centralized architecture where application data is stored outside of the React component tree. Any component, regardless of its position in the tree, can directly read from or write to this centralized storage without relying on props.

3. Redux Toolkit (RTK)

Redux Toolkit is the official, opinionated, and standard way to write Redux logic. It requires a strict, predictable architecture:

- **The Store:** The single, centralized JavaScript object that holds the entire global state of your application. Think of it as the application's global database.
- **The Slice (`createSlice`):** A modular segment of the Redux Store. Instead of one massive file, you divide the store into "slices" by feature (e.g., a `cartSlice`, a `userSlice`).
- **State:** The actual data stored inside the slice (e.g., an array of `cartItems`).
- **Actions & Reducers:**
    - **Action:** A descriptive payload that announces *what* happened (e.g., "ADD_TO_CART").
    - **Reducer:** A pure function that calculates *how* the state should change based on the Action. It takes the current state, applies the payload, and returns the new state.
- **Dispatch (`useDispatch`):** The hook used by a React component to send (dispatch) an Action to the Redux Store to trigger a state update.
- **Selector (`useSelector`):** The hook used by a React component to extract (read) specific pieces of data from the Redux Store.

4. Zustand

- **Definition:** A small, fast, and scalable bearbones state management solution built heavily around React Hooks.
- **Architecture Difference (Zustand vs. Redux):** Zustand eliminates the need for strict Action/Reducer boilerplate and Context Providers (`<Provider>`). You create a custom hook that directly holds both the state and the functions to update that state.

5. Context API Terminology (Native React)

The Context API is built directly into React to share data without passing props.

- **`createContext()`:** A built-in React function that creates a new "Context" object (the container for your global data).
- **Provider (`<Context.Provider>`):** A wrapper component generated by your Context object that "provides" the data to all nested components via a `value` prop.
- **`useContext()`:** A React Hook used by any child component to tap into the Provider and extract the data it needs.

5. Variants, Options, and Choices (When to use which?)

- **Redux Toolkit:**
  - *Pros: Strict architecture, excellent debugging tools (Redux DevTools), enterprise industry standard.*
  - *Cons: Requires more boilerplate code and setup steps.*
  - *Use Case:* Complex, large-scale features like Shopping Cart logic, complex user authentication, and heavy data transformation.
- **Zustand:**
  - *Pros:* Minimal boilerplate, extremely easy to set up, no <Provider> wrapping required.
  - *Cons:* Lacks the strict structural enforcement of Redux, which can lead to messy code in massive apps if not careful.
  - *Use Case:* Simpler global UI states like Light/Dark mode toggles, Sidebar open/close states, or user preferences.

Context API

- **Re-render Bottleneck:** By default, whenever the data inside a Context Provider changes, React forces *every single component* that consumes that context to re-render, even if they only needed a small piece of the data that didn't change.
- **Provider Hell:** The visual nesting problem that occurs in your `main.jsx` when you have to wrap your app in 5 or 6 different `<Context.Provider>` tags just to share different pieces of state.
- **Pros:**
  - **Native to React:** It is built directly into React v19. It requires zero external dependencies, meaning no extra `npm install` commands and no increase to your application's bundle size.
  - **Excellent Introductory Tool:** It is the easiest way to conceptually grasp the "Provider/Consumer" pattern before moving on to complex libraries like Redux.
- **Cons:**
  - **Performance Issues with Frequent Updates:** Because of the re-render bottleneck, it is highly inefficient for data that changes rapidly (like a user typing in a search bar or a rapidly updating shopping cart total).

    - **Strict Component Tree Boundary:** Because it relies on the `useContext` Hook, the data is trapped inside React components. You cannot read Context data inside plain JavaScript files (like an Axios configuration file).
    - **Requires Wrapper Boilerplate:** You are forced to wrap your application components inside `<Provider>` tags, which can lead to messy, deeply nested code (Provider Hell) as the app grows.
- **Use Case:**
    - Low-frequency, global "read-only" style data that rarely changes during a single user session. Excellent for Localization (Language switching between English and Arabic), basic User Profile details (like the user's name or avatar), or simple Auth boundaries where Axios interceptors are not required.